

Chapitre 1 – Au cœur de l'ordinateur

Notes sur le chapitre

Nous avons choisi d'ouvrir le manuel avec ce chapitre pour démystifier l'ordinateur en expliquant qu'il s'agit d'une machine très « bête » mais très rapide. Cependant, ce chapitre n'est pas un prérequis pour les chapitres suivants, jusqu'au chapitre 9 (des 0 et des 1) qui parle du codage binaire et des portes logiques. L'enseignant peut donc choisir de commencer directement par le chapitre 2 et couvrir le chapitre 1 plus tard.

Le simulateur utilisé dans l'activité d'introduction est le plus simple que nous avons pu trouver tout en présentant une interface relativement conviviale. Nous l'avons modifié de l'original (<https://github.com/c2r0b/vnmsim>) et adapté pour les besoins du manuel.

Les principales caractéristiques de ce simulateur sont :

- un seul registre, l'accumulateur (en plus du RI et du CP) ;
- des adresses mémoire symboliques (X, Y, Z, etc.) au lieu d'adresses numériques ;
- un jeu d'instructions minimal.

Le cours utilise le langage d'assemblage du simulateur (à l'exception de l'utilisation d'adresses mémoires numériques dans le premier exemple du cours et de l'instruction DEF ensuite, puisque le simulateur utilise seulement des adresses mémoire symboliques).

Les exemples de programmes du chapitre sont disponibles dans le menu Exemples du simulateur :

- ADDITION (activité d'introduction et cours)
- EQUAL (activité d'introduction)
- MAX (cours)
- PUISSANCE (cours)

Le programme de l'exercice 5 du QCM est disponible dans le fichier `qcm5.json`. Les programmes des autres exercices sont également disponibles dans des fichiers de nom `exoNN.json` où NN est le numéro de l'exercice. Ces programmes peuvent être chargés dans le simulateur par simple cliquer-tirer du fichier vers la fenêtre du simulateur, ou bien par la commande Ouvrir du menu Projet du simulateur. Le chargement d'un programme prend parfois un peu de temps.

À titre d'information, il existe de nombreux autres simulateurs (la grande majorité en anglais), par exemple :

- Little Man Computer - <https://www.101computing.net/LMC/#>
- AQA Assembly Language Simulator by Peter Higginson - <http://www.peterhigginson.co.uk/AQA/>
- OCR A Level CPU Simulator - <https://tools.withcode.uk/cpu/>

Activité : Simulateur de machine de Von Neumann

Exécuter un programme

2. Le Registre d'Instruction (RI) contient l'instruction en cours d'exécution. Son contenu vient du contenu de la mémoire à l'adresse du Compteur de Programme (CP). Le CP valait 0 au début, le contenu de la mémoire à l'adresse 0 est l'instruction `LOD X` qui est chargée dans le RI.

L'exécution de cette instruction va chercher le contenu de la mémoire pour la cellule X, qui vaut 2. Celle-ci est envoyée dans l'entrée de l'Unité Arithmétique et Logique (UAL), qui la stocke dans l'accumulateur (ACC).

À l'issue de l'étape, le CP vaut 1 (adresse de la prochaine instruction) et l'accumulateur vaut 2 (valeur chargée par l'instruction `LOD X`).

3. Le chiffre 2 dans l'entrée gauche de l'UAL vient de l'accumulateur ACC.

L'instruction qui a été exécutée à cette étape est `ADD Y`, qui ajoute à cette valeur 2 la valeur de la cellule mémoire Y.

Cette instruction a chargé dans l'autre entrée de l'UAL la valeur de Y (5), et effectué l'addition avec 2, d'où le résultat 5 dans l'accumulateur.

4. L'instruction exécutée à cette étape est `STO Z`, qui enregistre la valeur de l'accumulateur (5) dans la cellule mémoire Z.

Le compteur de programme vaut 3 à l'issue de cette étape. L'exécution de cette instruction va arrêter le programme.

Les sauts

1. L'instruction `JMP 1` met la valeur 1 dans le compteur de programme CP, de telle sorte que l'instruction qui est exécutée ensuite est l'instruction à l'adresse 1 au lieu de celle à l'adresse 4.

Le programme « boucle », c'est-à-dire qu'il exécute les instructions des adresses 1 (`ADD Y`) et 2 (`STO Z`) de manière répétitive. La valeur de la cellule mémoire Z augmente donc de 2 à chaque instruction.

Le programme ne s'arrête pas : il exécute une boucle infinie.

2. Non, le programme n'a pas exécuté la ligne 3. La valeur de Z est 0 (instruction `STO Z` à l'adresse mémoire 4).

3. Oui, cette fois-ci le programme a exécuté l'instruction à l'adresse 3 (`LOD #1`) et stocké son résultat (1) dans la cellule mémoire Z. Z vaut donc 1.

4. L'instruction `JMZ n` effectue un saut à l'adresse n si l'accumulateur contient une valeur différente de 0, sinon elle ne fait rien et on passe à l'instruction suivante.

Le programme met 0 dans Z si X est égale à Y, et 1 sinon. Il teste donc l'égalité de deux variables.

QCM (CHECKPOINT)

1 a et d ; 2 b ; 3 b et c ; 4 b et c ; 5 d ; 6 b ; 7 a ; 8 b ; 9 a et d.

EXERCICES

Exercices Newbie

1 • Le contenu du compteur de programme est envoyé à la mémoire.

- L'instruction est lue de la mémoire.
- Le compteur de programme est incrémenté.
- Le code opération est envoyé à l'UAL.
- L'adresse du code opération est envoyée à la mémoire.
- Le contenu de la mémoire est envoyé à l'UAL.
- L'UAL effectue l'opération entre l'accumulateur et la donnée lue de la mémoire.

2

RI	ACC	RI	CP	Donnée	Adresse
LOAD X	X			X	X
LOAD #1	X				
ADD Y	X			X	X
MUL #2	X				
STORE Z				X	X
JMP a			X		
JMZ b			X		

3 a. Pour $X = 10$, $Y = 15$ et $Z = 20$, le programme met 50 dans W.

Pour $X = 10$, $Y = 20$ et $Z = 20$, il met 100 dans W.

Pour $X = 10$, $Y = 12$ et $Z = 20$, il met 20 dans W.

b. Le programme met dans W la proportion de Y par rapport à l'intervalle $[X, Z]$, en pourcentage. Dans le premier cas 15 est au milieu de $[10, 20]$, donc W vaut 50 %. Dans le second cas, 10 est le début de l'intervalle $[10, 20]$, donc W vaut 0 %. Dans le dernier cas, 12 représente 20 % de l'intervalle $[10, 20]$.

Si on analyse le programme on voit qu'il calcule d'abord $W = X - Z$ (trois premières lignes), puis il calcule $(X - Y) * 100$ (trois lignes suivantes), qu'il divise par W pour stocker le résultat dans W (deux lignes suivantes). On peut donc l'écrire sous la forme :

ranger la valeur de $(X - Z)$ dans W
ranger la valeur de $(X - Y) * 100 / W$ dans W

soit

ranger la valeur de $(X - Y) * 100 / (X - Z)$ dans W

qui est la règle de trois pour calculer une proportion.

c. Le programme est disponible dans le fichier `exo3.json` et peut être chargé dans le simulateur par cliquer-tirer.

4

0 JMP 0

Le programme exécute l'instruction `JMP 0`, qui est à l'adresse 0. Il exécute donc cette instruction indéfiniment. Si une boucle de ce genre se retrouve dans un programme, on n'a en général pas d'autre choix que de redémarrer l'ordinateur.

5 a.

```
LOAD X
STORE Z ; mettre la valeur de X dans Z
LOAD Y
STORE X ; mettre la valeur de Y dans X
LOAD Z
STORE Y ; mettre la valeur de Z dans X
HALT
```

b. Le programme est disponible dans le fichier `exo5.json` et peut être chargé dans le simulateur par cliquer-tirer.

6 a.

```
LOAD X
DIV #2
MUL #2
CMP X
JMZ a ; Saut à l'adresse a si X est nul
LOAD #1 ; Charger 1
JMP b
a: LOAD #0 ; Charger 0
b: STORE Y
HALT
```

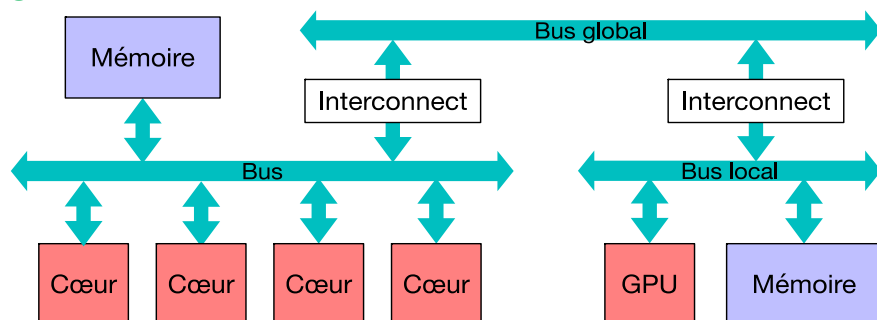
b. Le programme est disponible dans le fichier `exo6.json` et peut être chargé dans le simulateur par cliquer-tirer.

7 a.

```
LOAD X
CMP Y ; comparer X et Y
JMPN ymax ; aller à ymax si X < Y
LOAD X
STORE W ; ranger X dans W
JMP suite
ymax: LOAD Y
STORE W ; ranger Y dans W
suite: CMP Z ; comparer le max de X et Y à Z
JMPP fin ; si max(X, Y) > Z on a fini
LOAD Z
STORE W ; ranger Z dans W
fin: HALT
```

b. Le programme est disponible dans le fichier `exo7.json` et peut être chargé dans le simulateur par cliquer-tirer.

8



9

Pour traiter 1 million de données il faut 100 millions d'instructions. Le processeur exécute 1 milliard d'instructions par seconde (1 Kilo = 1000, 1 Mega = 1 million, 1 Giga = 1 milliard). Il faut donc 1/10ème de seconde pour traiter les données.

10 a.

- livre : $1500 \times 300 = 450 \text{ Ko}$
- photo : $4000 \times 3000 \times 4 = 48 \text{ Mo}$
- vidéo (5 mins) : $20 \times 5 = 100 \text{ Mo}$
- film (2 h) : $20 \times 120 = 2,4 \text{ Go}$

b.

- $500 \text{ Go} / 450 \text{ Ko} = 1\,111\,111 \text{ livres}$
- $500 \text{ Go} / 48 \text{ Mo} = 10\,416 \text{ photos}$
- $500 \text{ Go} / 100 \text{ Mo} = 5\,000 \text{ vidéos}$
- $500 \text{ Go} / 2,4 \text{ Go} = 208 \text{ films}$

Exercices Initié

11 a. L'Unité Arithmétique et Logique est en rouge au milieu (ALU pour Arithmetic and Logic Unit). Ses entrées sont deux registres : l'accumulateur en haut à gauche (« Accumulator ») et un registre temporaire juste à sa droite (« Temp Register »). Le registre « Accumulator Latch » permet de stocker temporairement le contenu de l'accumulateur.

Le registre d'instruction (« Instruction Register ») est un peu plus loin à droite du registre temporaire. Comme les deux autres registres, il est connecté au bus de données en haut (« 8 Bit internal Data Bus »). Il est également connecté à l'unité de contrôle en bas.

Le module « Multiplexer » à droite contient d'autres registres, notamment le compteur de programme (« Program Counter »). Ce module est connecté au bus de données en haut (« D0-D7 bidirectional Data Bus ») et au bus d'adresses en bas à droite (« A0-A15 Address Bus »). On note que le bus de données a 8 bits et le bus d'adresse a 16 bits. On peut donc adresser 2^{16} octets, soit 64 kilo-octets.

L'unité de contrôle est en bas (« Timing and Control ») : elle reçoit le décodage de l'instruction (flèche grise) et elle est connectée aux différents organes du processeur (flèches noires).

b. Non, la mémoire n'apparaît pas dans ce diagramme. Elle est connectée au bus de donnée en haut (« D0-D7 bidirectional Data Bus ») et au bus d'adresse en bas à droite (« A0-A15 Address Bus »).

12 a. $2 \text{ MHz} / (4 + 11) / 2 = 266\,666$ instructions par seconde en moyenne.

b. $20\,000\,000 / 266\,666 = 75$

c. $10^{12} / 266\,666 = 3\,750\,000$

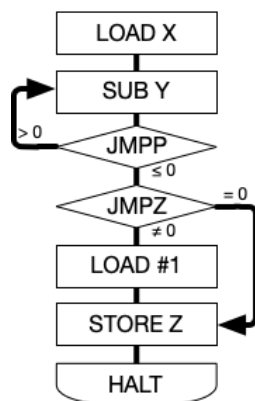
13 a.

```
LOAD #0
STORE Z ; mettre 0 dans Z
b: LOAD Z
ADD X
STORE Z ; ajouter X à Z
LOAD X
SUB #1
STORE X ; enlever 1 de X
JMPP b ; boucler si X > 0
HALT
```

Ce programme est disponible dans le fichier `exo13.json` et peut être chargé dans le simulateur par cliquer-tirer.

b. $3 + 7 \times (n + 1)$ instructions.

14 a.



b.

- $X = 10$ et $Y = 3$: Z vaut 1
- $X = 10$ et $Y = 2$: Z vaut 0

c. Z vaut 0 si X est divisible par Y , sinon il vaut 1.

d. Ce programme est disponible dans le fichier `exo14.json` et peut être chargé dans le simulateur par cliquer-tirer.

15 a.

```
LOAD X
STORE Z ; copier X dans Z
b: LOAD Z
CMP Y
JMPN a ; si Z < Y, on a fini
LOAD Z
```

```

SUB Y
STORE Z ; retirer Y de Z
JMP b ; boucler
a: HALT

```

Ce programme est disponible dans le fichier `exo15a.json` et peut être chargé dans le simulateur par cliquer-tirer.

b.

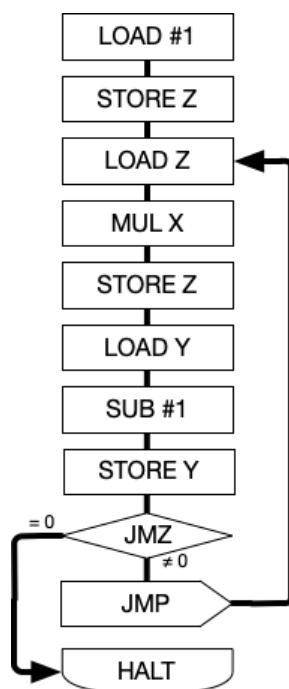
```

LOAD X
STORE Z ; copier X dans Z
LOAD #0
STORE W ; mettre 0 dans W (quotient)
b: LOAD Z
CMP Y
JMPN a ; si Z < Y, on a fini
LOAD Z
SUB Y
STORE Z ; retirer Y de Z
LOAD W
ADD #1
STORE W ; ajouter 1 au quotient
JMP b ; boucler
a: HALT

```

Ce programme est disponible dans le fichier `exo15b.json` et peut être chargé dans le simulateur par cliquer-tirer.

16 a.



b.

- $X = 2$ et $Y = 8$: Z vaut 256
- $X = 3$ et $Y = 4$: Z vaut 81

c. $Z = X^Y$

d. Ce programme est disponible dans le fichier `exo16.json` et peut être chargé dans le simulateur par cliquer-tirer.

17 a. Le processeur exécute $1 \text{ MHz} / 5 = 200\,000$ instructions par seconde. La boucle d'attente nécessite deux instructions (une pour compter, une pour le saut), il faut donc l'exécuter 100 000 fois.

```
def LED 1000
go: LOAD #100
  STORE LED ; allumer la LED
  LOAD #100000 ; commencer le compte à rebours
on: SUB #1 UN
  JMPP on ; boucle du compte à rebours
  LOAD #0
  STORE LED ; éteindre la LED
  LOAD #100000 ; commencer le compte à rebours
off: SUB #1
  JMPP off ; boucle du compte à rebours
  JMP go ; recommencer clignotement
```

Une version de ce programme adaptée au simulateur est dans le fichier `exo17a.json`. Elle utilise la variable `N` au lieu de `LED`, et le compte à rebours commence à 10 au lieu de 100 000 car le simulateur est très lent par rapport à un vrai ordinateur (même avec le délai à 0 ms).

b. Il faut remplacer la ligne 4 par `LOAD X` et la ligne 9 par `LOAD Y`. Si les valeurs sont exprimées en millisecondes, il faut les multiplier par 100 (`MUL #100`) avant chaque boucle du compte à rebours.

c. L'intensité de la LED doit passer de 0 à 100 puis de 100 à 0. On crée deux boucles imbriquées : la boucle interne compte de 1 000 à 0, la boucle externe fait varier l'intensité de la LED.

```
def LED 1000
LOAD #0
STORE LED ; initialiser la LED
on: LOAD #1000 ; démarrer compte à rebours
cr1: SUB #1
  JMPP cr1 ; boucle du compte à rebours
  LOAD LED
  ADD #1
  STORE LED ; augmenter intensité LED
  CMP #100 ; tester intensité maximale
  JMPN on ; boucler si non atteinte
off: LOAD #1000 ; démarrer compte à rebours
cr2: SUB #1
  JMPP cr2 ; boucle du compte à rebours
  LOAD LED
  SUB #1
  STORE LED ; diminuer intensité LED
  JMPP off ; boucler si positive
  JMP on ; recommencer
```

Une version de ce programme adaptée au simulateur est dans le fichier `exo17c.json`. Elle utilise la variable `N` au lieu de `LED`, et le compte à rebours commence à 2 au lieu de 1 000 car le simulateur est très lent par rapport à un vrai ordinateur (même avec le délai à 0ms).

18

- $50 \cdot 10^{-6} / 10 \cdot 10^{-9} = 5 \cdot 10^3$, soit 5 000 accès mémoire pour un accès disque SSD ;
- $10 \cdot 10^{-3} / 10 \cdot 10^{-9} = 1 \cdot 10^6$, soit 1 million d'accès mémoire pour un accès disque mécanique.

19 La table est issue de la page Wikipedia https://en.wikipedia.org/wiki/Moore%27s_law.

On peut produire le graphe à l'aide du code ci-dessous.

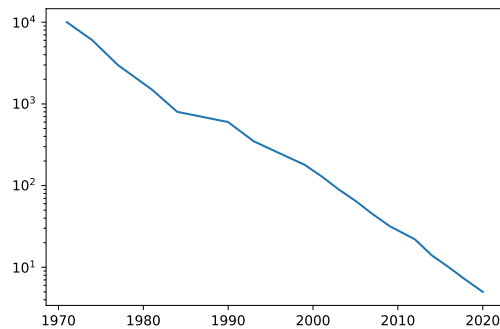
Un graphe similaire est visible ici :

https://upload.wikimedia.org/wikipedia/commons/c/c7/Comparison_semiconductor_process_nodes.svg

On observe que la diminution est pratiquement linéaire dans ce graphe. Comme l'échelle verticale est logarithmique, cela signifie que la diminution est exponentielle : elle est divisée par un facteur constant chaque année.

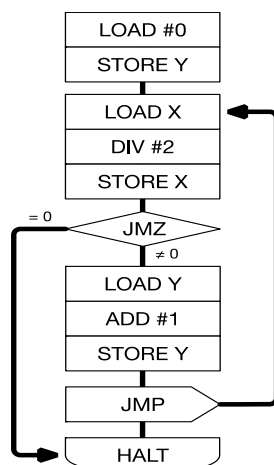
```
x = [1971, 1974, 1977, 1981, 1984, 1990, 1993, 1996, 1999, 2001, 2003, 2005, 2007, 2009, 2012, 2014, 2016, 2018, 2020]
y = [10000, 6000, 3000, 1500, 800, 600, 350, 250, 180, 130, 90, 65, 45, 32, 22, 14, 10, 7, 5]
```

```
from matplotlib.pyplot import plot, yscale, show
yscale("log")
plot(x, y)
show()
```



Exercices Titan

20 a.



b. Pour $X = 8$, Y vaut 3.

Pour $X = 15$, Y vaut 3.

Pour $X = 16$, Y vaut 4.

Chapitre 1 – Au cœur de l'ordinateur

c. Le programme calcule le nombre de fois que l'on peut diviser x par 2 jusqu'à ce que le résultat soit 0. Cela correspond à la partie entière du logarithme en base 2, qui est aussi le nombre de bits nécessaires pour représenter x en base 2.

d. Le programme est disponible dans le fichier `exo20.json` et peut être chargé dans le simulateur par cliquer-tirer.

21

LOAD Z ; *charger l'opérateur*

JMZ add

SUB #1

JMZ sub

SUB #1

JMZ mul

SUB #1

JMZ div

HALT

add: LOAD X

ADD Y

JMP fin

sub: LOAD X

SUB Y

JMP fin

mul: LOAD X

MUL Y

JMP fin

div: LOAD X

DIV Y

JMP fin

fin: STORE R

HALT

Le programme commence par charger Z et selon sa valeur, branche sur l'une des quatre sections étiquetées add (addition), sub (soustraction), mul (multiplication) ou div (division). Chaque section effectue l'opération entre X et Y et va à l'étiquette fin où le résultat est stocké dans R et le programme s'arrête.

Le programme est disponible dans le fichier `exo21.json` et peut être chargé dans le simulateur par cliquer-tirer.

22

LOAD N

STORE R ; $R \leftarrow N$

DIV #2

STORE Z ; $Z \leftarrow N/2$

a: LOAD Z

JMZ fin ; *fin si Z=0*

SUB #1

STORE Z ; $Z \leftarrow Z-1$

LOAD N

DIV R

ADD R

DIV #2

STORE R ; $R \leftarrow (R + N/R) / 2$

JMP a ; *boucle*

fin: HALT

On utilise l'adresse mémoire Z pour compter le nombre de fois que l'on doit faire la boucle.

Le programme est disponible dans le fichier `exo22.json` et peut être chargé dans le simulateur par cliquer-tirer.

23 a. L'écran contient $1\,920 \times 1\,080 = 2\,073\,600$ pixels, soit $2\,073\,600 \times 3 = 6\,220\,800$ octets. Un mega-octet (Mo) est égal à un million d'octets, le contenu de l'écran nécessite donc un peu plus de 6,2 Mo.

b. $3\,840 \times 2\,160 \times 3 = 24\,883\,200$ octets, soit près de 25 Mo (4 fois plus que pour la question **a.**).

c. 1 GHz représente un milliard d'opérations par seconde. Dans le premier cas, une image est affichée est $1\,000\,000\,000 / 2\,073\,600 = 482,253$, soit plus de 480 images par seconde.

Dans le second cas $1\,000\,000\,000 / (3\,840 \times 2\,160) = 120,563$, soit 120 images par seconde.

La fréquence de réaffichage habituelle d'un écran de bonne qualité est 60 Hz (60 images par seconde). On voit que dans le second cas le processeur passerait la moitié de son temps à afficher l'écran. C'est la raison pour laquelle on utilise des processeurs graphiques (GPUs) qui ont des capacités d'affichage beaucoup plus élevées, et qui déchargent le processeur central (CPU).

24 a. Si chaque processeur accède la mémoire une instruction sur deux en moyenne, la probabilité que les deux accèdent la mémoire en même temps est de 25 %.

En effet il y a quatre cas de figure qui ont chacun la même probabilité : les deux processeurs veulent accéder à la mémoire en même temps, ou bien seul le processeur A accède à la mémoire, ou bien seul le processeur B, ou bien aucun des deux.

b. Si le programme a n instructions, 25 % de celles-ci (en moyenne) vont créer un conflit d'accès, soit $n/4$. A chaque conflit, l'un des processeurs « gagne » l'accès à la mémoire, et l'autre doit réessayer à l'instruction suivante. Chaque processeur devra donc attendre pour accéder à la mémoire la moitié du nombre de conflits, soit $n/8$.

Cependant, après un tel conflit, il y a une chance sur deux que l'autre processeur accède à nouveau à la mémoire et crée un nouveau conflit. Le processeur va « perdre » ce conflit la moitié du temps, et donc attendre à nouveau dans un quart des cas, soit $(n/8)/4 = n/32$ cas. Il faut aussi prendre en compte la possibilité d'un nouveau conflit, à nouveau dans un quart des cas, soit $(n/32)/4 = n/128$, et ainsi de suite, mais les probabilités deviennent négligeables pour les besoins de cette analyse.

En considérant seulement ces trois cas, un programme de n instructions mettra donc en moyenne $n + n/8 + n/32 + n/128$ instructions à s'exécuter, soit un ralentissement de $(1/8 + 1/32 + 1/128) \times 100 = 16,4$ %.

c. D'après la question précédente, chaque programme mettra environ 1,16 secondes, et comme ils s'exécutent en même temps, c'est aussi le temps total d'exécution.

Ce temps est à comparer à 2 secondes pour exécuter les deux programmes à la suite l'un de l'autre sur un mono-cœur. L'utilisation d'un bi-cœur permet donc un gain de 42 %.

d. Si on considère 4 programmes qui s'exécutent en parallèle sur 4 cœurs, chacun accédant à la mémoire une instruction sur deux en moyenne, on va saturer l'accès à la mémoire puisque, en moyenne, 2 programmes voudront accéder à la mémoire en même temps à chaque cycle d'instruction. Il en résulte que le temps d'exécution total ne peut pas être inférieur à deux fois le temps d'exécution de chaque programme.

Les quatre programmes qui prendraient chacun 1 seconde pour s'exécuter nécessiteront donc au moins 2 secondes en tout sur un quadricœurs, au lieu de 4 secondes si on les exécutait l'un après l'autre, soit une accélération de 50 % au maximum.

25 a. En première approximation, chaque 10 instructions, le processeur doit exécuter l'équivalent de deux instructions supplémentaires, soit un ralentissement de 20 %.

Cependant, un seul processeur peut utiliser le bus à la fois, donc s'il y a deux processeurs qui utilisent le bus chacun 20 % du temps, le risque de conflit est $0,2 * 0,2 = 0,04$, soit 4 %. Le ralentissement sera donc plutôt de 24 %.

b. Chaque programme mettra environ 1,24 secondes et ils s'exécutent en même temps, donc le temps total est 1,24 secondes au lieu de 2 secondes si les deux programmes s'exécutaient l'un après l'autre, soit un gain de 38 %.

c. Ici aussi, en première approximation, chaque programme est ralenti de 20 %. Cependant, la probabilité de conflit d'accès au bus est plus élevée qu'avec 2 processeurs.

En effet, considérons la table suivante où chaque colonne A, B, C, D représente un processeur et chaque ligne une configuration possible : 0 = pas d'accès à la mémoire, 1 = accès à la mémoire.

A	B	C	D	Conflit d'accès
0	0	0	0	non
0	0	0	1	non
0	0	1	0	non
0	0	1	1	oui (2 accès)
0	1	0	0	non
0	1	0	1	oui (2 accès)
0	1	1	0	oui (2 accès)
0	1	1	1	oui (3 accès)
1	0	0	0	non
1	0	0	1	oui (2 accès)
1	0	1	0	oui (2 accès)
1	0	1	1	oui (3 accès)
1	1	0	0	oui (2 accès)
1	1	0	1	oui (3 accès)
1	1	1	0	oui (3 accès)
1	1	1	1	oui (4 accès)

On compte 6 combinaisons sur 16 avec deux accès simultanés, 4 combinaisons avec 3 accès simultanés, et 1 combinaison avec 4 accès simultanés. Le risque de conflit est donc $6 * 0,2 * 0,2 + 4 * 0,2 * 0,2 * 0,2 + 1 * 0,2 * 0,2 * 0,2 * 0,2 = 0,2736$, soit un peu plus de 27 %.

Le temps total d'exécution des 4 programmes en parallèle est donc 1,27 secondes, au lieu de 4 secondes s'ils s'exécutaient sur un mono-processeur.